CS 4341

Exam 2 Study Guide

Assigned-reading summaries and worked examples

Six syllabus chapters: Chapter 1 From Zero to One; Chapter 2 Combinational Logic; Chapter 3 Sequential Logic; Chapter 5 Digital Building Blocks; Chapter 6 Architecture; Chapter 7 Microarchitecture.

Prepared November 3, 2026. Follows the structure and visual theme of the supplied CS 3345 study guide. Each numbered topic has a concise reading summary and a worked example; original examples are labeled and calculations checked.

HOW TO USE THIS GUIDE

Study one topic's summary, then solve the worked example before viewing its solution steps. Check the exam-ready distinction, attempt the independent problem, and revisit the topic on another day. Use the last pages as an open-notes index rather than a substitute for practice.

Highest priority: Ch 6 MIPS encoding/decoding, program translation, arrays, stack frames, and Ch 7 instruction datapaths, control signals, CPI, pipeline timing, and hazards. Earlier chapters are still on the cumulative exam.

Exam: November 12, 2026. The Fall 2026 syllabus lists Exam 2 as Chapters 1, 2, 3, 5, 6, 7; closed book and open notes. Chapter 8 is not in the listed Exam 2 scope.

Source base: the uploaded Fall 2026 syllabus, professor's Ch 1/2/3/5/6/7 lecture slides, Harris & Harris (2nd ed.), eLearning Ch 6/7 objectives, and the sample 4341 exam. Source scope is documented on the last page.

Contents

Find a topic, then its worked example on the next page

Page numbers below refer to this study guide. The textbook references within sections use printed textbook pages, not PDF-viewer offsets.

01 Number Systems and Two's Complement 3
02 Logic Levels, Gates, and Noise Margins 5
03 Boolean Algebra, SOP, and POS 7
04 Karnaugh Maps and Don't-Cares 9
05 Multiplexers, Decoders, and Gate Delay 11
06 Latches, Flip-Flops, and Registers 13
07 Finite-State Machines 15
08 Sequential Timing and Parallelism 17
09 Adders, ALUs, and Carry Delay 19
10 Fixed Point and IEEE 754 Floating Point 21
11 Counters, Registers, and Memory Arrays 23
12 MIPS Registers, Operands, and Memory 25
13 Byte Ordering and Signed Immediate Values 27
14 R-Type, I-Type, and J-Type Encoding 29
15 MIPS Logic, Shifts, and Constants 31
16 C Conditionals and Loops in MIPS 33
17 MIPS Arrays and Index Arithmetic 35
18 Functions, Stack, and Recursion 37
19 Addressing Modes and Stored Programs 39
20 Microarchitecture and Performance Measures 41
21 Single-Cycle Datapath and Control 43
22 Single-Cycle Critical-Path Timing 45
23 Multicycle Datapath and CPI 47
24 Pipelining and Stage Timing 49
25 RAW Data Hazards, Forwarding, and Stalls 51
26 Control Hazards and Branch Prediction 53
Reference 1: Cumulative essentials 55
Reference 2: MIPS encoding and instruction index 56
Reference 3: Microarchitecture control and performance 57
Reference 4: Exam-ready distinctions and formula index 58
Practice: mixed questions 59
Practice: solutions and explanations 60
Sources and attribution 61

01 Number Systems and Two's Complement

Binary, hexadecimal, signed ranges, and overflow

Slides Ch 1: 11-46 | Harris & Harris, pp. 9-19

Base conversion

An N-bit binary word encodes 2^N different patterns. Convert binary to decimal using powers of two; convert binary to hexadecimal by grouping from the right into four-bit nibbles. Each hex digit represents exactly four bits, so 0x2D = 0010 1101.

Signed representations

Unsigned N-bit values span 0 to 2^N-1. Sign/magnitude uses the highest bit for sign and has both +0 and -0. Two's complement spans -2^(N-1) through 2^(N-1)-1 and has a unique zero. Negate an N-bit value by inverting all bits and adding one.

Arithmetic and overflow

Discard carry-out beyond N bits. For unsigned addition, a carry-out indicates overflow. For signed two's-complement addition, overflow occurs when same-sign operands produce a result of the opposite sign. Carry-out by itself is not the signed-overflow test.

Unsigned: 0 ... 2^N - 1 | Two's complement: -2^(N-1) ... 2^(N-1)-1

EXAM-READY DISTINCTION

Do not confuse a leading 1 with an inherently negative bit pattern: interpretation depends on whether the number is signed.

01 Number Systems and Two's Complement

Worked example and independent test

Slides Ch 1: 11-46 | Harris & Harris, pp. 9-19

PROBLEM AND ASSUMPTIONS

Original: encode -13 as an 8-bit two's-complement number; add 100 and 45 using 8-bit arithmetic.

Solution steps

1. 13 decimal is 00001101 binary. Invert to 11110010; add one to obtain **11110011** (0xF3).
2. 100 = 01100100 and 45 = 00101101. Binary addition gives 10010001 (0x91).
3. The 8-bit signed result has its leading bit set and represents -111; 100 and 45 are both positive, so signed overflow occurred. The mathematically correct sum 145 is outside -128...127.

Quantity	Binary (8 bits)	Interpretation
13	00001101	+13
-13	11110011	-13
100 + 45	10010001	-111 if signed; 145 modulo 256

WHAT THIS EXAMPLE PROVES

The same bits can represent different values in unsigned and signed formats; signed overflow tests operand and result signs.

Try independently

Try independently: represent -37 in 8 bits and determine whether 0x7F + 0x01 overflows signed 8-bit arithmetic.

02 Logic Levels, Gates, and Noise Margins

Digital interpretation of physical voltages

Slides Ch 1: 47-62 | Harris & Harris, pp. 19-25

Core gate meanings

AND gives 1 only if every input is 1. OR gives 1 when at least one is 1. XOR gives 1 for an odd number of asserted inputs. NAND, NOR, and XNOR are inverted forms. A bubble on a symbol indicates inversion.

Driver and receiver thresholds

A driver promises a LOW at or below V_OL and a HIGH at or above V_OH. A receiver recognizes a LOW at or below V_IL and a HIGH at or above V_IH. Between V_IL and V_IH is an indeterminate input region, not an extra logic state.

Static discipline

Compatible logic families ensure valid outputs become valid inputs even with bounded noise. A buffer can strengthen a signal or drive more loads although its Boolean function is simply Y=A.

NML = V_IL - V_OL | NMH = V_OH - V_IH

EXAM-READY DISTINCTION

Output thresholds (V_OH and V_OL) belong to the driver. Input thresholds (V_IH and V_IL) belong to the receiver.

02 Logic Levels, Gates, and Noise Margins

Worked example and independent test

Slides Ch 1: 47-62 | Harris & Harris, pp. 19-25

PROBLEM AND ASSUMPTIONS

Textbook-style original: V_IL=0.8 V, V_IH=2.0 V, V_OL=0.4 V, V_OH=2.4 V. Find both noise margins.

Solution steps

1. LOW margin: V_IL - V_OL = 0.8 - 0.4 = **0.4 V**.
2. HIGH margin: V_OH - V_IH = 2.4 - 2.0 = **0.4 V**.
3. Both valid levels have a 0.4 V guarantee against worst-case noise in the unfavorable direction. Noise beyond that may push the receiver input into the undefined region.

Gate	A=0,B=0	0,1	1,0	1,1
AND	0	0	0	1
OR	0	1	1	1
XOR	0	1	1	0
NAND	1	1	1	0
NOR	1	0	0	0
XNOR	1	0	0	1

WHAT THIS EXAMPLE PROVES

Noise margin measures tolerable disturbance, not the voltage difference between V_IH and V_IL.

Try independently

Given V_IL=1.35, V_OL=0.33, V_OH=3.84, V_IH=3.15, calculate NML and NMH. Which is smaller?

03 Boolean Algebra, SOP, and POS

Turning truth tables into valid equations

Slides Ch 2: 9-36 | Harris & Harris, pp. 58-70

Canonical forms

A minterm is an AND of every input variable or its complement, true for one input row. Sum-of-products (SOP) ORs minterms for rows where F=1. A maxterm is an OR containing every variable; product-of-sums (POS) ANDs maxterms for rows where F=0.

Useful identities

Identity: A+0=A and A·1=A. Complement: A+A'=1, A·A'=0. Absorption: A+AB=A. De Morgan: (AB)'=A'+B', (A+B)'=A'B'. Distribution works in both directions.

From equations to circuits

Write a truth table or apply Boolean algebra to verify each transformation. NAND-only and NOR-only circuits exploit inversion bubbles and De Morgan, but note how many inversions are required.

F = Σm(indices with F=1) | F = ΠM(indices with F=0)

EXAM-READY DISTINCTION

SOP uses the 1-rows; POS uses the 0-rows. Canonical expressions are not necessarily simplified expressions.

03 Boolean Algebra, SOP, and POS

Worked example and independent test

Slides Ch 2: 9-36 | Harris & Harris, pp. 58-70

PROBLEM AND ASSUMPTIONS

Original: F(A,B,C)=Σm(1,3,5,7). Find a minimal SOP and canonical POS.

Solution steps

1. The 1-rows are binary 001, 011, 101, 111. In all these rows C=1 while A and B vary.
2. Therefore the minimized SOP is **F=C**. The canonical SOP contains four distinct 3-literal minterms.
3. The 0-rows are 000, 010, 100, 110, so canonical POS = (A+B+C)(A+B'+C)(A'+B+C)(A'+B'+C).

A	B	C	F
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	0
1	1	1	1

WHAT THIS EXAMPLE PROVES

All 1-minterms differ only in A and B, so those variables disappear in the simplest equation.

Try independently

Try F(A,B,C)=Σm(0,2,4,6). What is the simplified SOP? Write one corresponding maxterm.

04 Karnaugh Maps and Don't-Cares

Use adjacency to remove changing variables

Slides Ch 2: 43-56 | Harris & Harris, pp. 75-82

K-map layout

Rows and columns use Gray-code order (00,01,11,10) so neighboring cells differ in exactly one variable. Left/right edges and top/bottom edges also touch. Corners may form a wrap-around group.

Grouping

For SOP, group 1s in rectangles containing 1,2,4,8,... cells. Every group must have power-of-two area, with wraparound allowed. Prefer large groups and cover every 1; overlap is permitted. Variables changing within a group are eliminated.

Don't-cares

X cells can be used as 1 if this simplifies a group, or ignored if not. Never require a group to cover X. Grouping zeros instead yields a POS form.

For SOP: group 1-cells; each unchanged 1 stays uncomplemented and unchanged 0 stays complemented.

EXAM-READY DISTINCTION

An adjacent pair is defined by one changing bit, not by proximity in ordinary binary counting order.

04 Karnaugh Maps and Don't-Cares

Worked example and independent test

Slides Ch 2: 43-56 | Harris & Harris, pp. 75-82

PROBLEM AND ASSUMPTIONS

Original: simplify F(A,B,C,D)=Σm(1,3,4,6,9,11,12,14) by a 4-variable K-map.

Solution steps

1. Place AB=00,01,11,10 on rows and CD=00,01,11,10 on columns.
2. Group minterms {1,3,9,11} (all B=0 and D=1) to get B'D.
3. Group {4,6,12,14} (all B=1 and D=0) to get BD'. These two implicants cover every listed 1, so **F=B'D+BD'**.

AB \ CD	00	01	11	10
00	0	1	1	0
01	1	0	0	1
11	1	0	0	1
10	0	1	1	0

WHAT THIS EXAMPLE PROVES

This pattern is XOR of B and D. The ordering of A and C is irrelevant to the simplified function.

Try independently

Try changing minterm 14 to a don't-care. Does the same two-term expression still work? Can any better grouping exist?

05 Multiplexers, Decoders, and Gate Delay

Selection circuits, one-hot outputs, critical paths

Slides Ch 2: 57-67 | Harris & Harris, pp. 83-95

Multiplexers

A 2:1 MUX selects D0 when S=0 and D1 when S=1: Y=S'D0+SD1. An n-select-bit MUX chooses one of 2^n data inputs. Boolean functions may be implemented by choosing variables as selectors and using constants or other variables for data inputs.

Decoders

An n-to-2^n decoder asserts one uniquely selected output for each binary input combination, usually under an enable. An active-low output is asserted at 0. Check the diagram's active-high/active-low markings.

Timing paths

Propagation delay t_pd is the latest an output becomes valid after an input change; contamination delay t_cd is the earliest output may change. The critical path is the largest end-to-end propagation-delay sum, not necessarily the path with the most drawn gates.

2:1 MUX: Y=S'D0+SD1 | longest propagation path = sum of t_pd along that path

EXAM-READY DISTINCTION

A 4:1 MUX has four DATA lines but only two SELECT bits. Decoder outputs are one-hot unless polarity/enable changes this.

05 Multiplexers, Decoders, and Gate Delay

Worked example and independent test

Slides Ch 2: 57-67 | Harris & Harris, pp. 83-95

PROBLEM AND ASSUMPTIONS

Original: a 4:1 MUX uses S1S0=10 and data inputs D0=0, D1=1, D2=A, D3=A'. What is Y?

Solution steps

1. The selector value 10 represents index two, so the MUX passes **D2**. Therefore Y=A regardless of the other three data inputs.
2. For a 2-to-4 active-high decoder with enable 1, the same input 10 activates output Y2 only.
3. For timing, suppose one path contains gates of 2 ns, 5 ns, 3 ns and another contains 4 ns, 4 ns. Their propagation totals are 10 ns and 8 ns. Critical-path delay is **10 ns**.

S1S0	Selected data	Decoder one-hot output
00	D0	0001
01	D1	0010
10	D2	0100
11	D3	1000

WHAT THIS EXAMPLE PROVES

Do not infer which output is active from physical pin order without inspecting the naming and polarity.

Try independently

Build Y=A XOR B using a 2:1 MUX with S=A. Determine D0 and D1 as functions of B.

06 Latches, Flip-Flops, and Registers

State held by levels versus clock edges

Slides Ch 3: 3-28 | Harris & Harris, pp. 109-122

Combinational versus sequential

A combinational circuit depends only on present inputs. Sequential logic also depends on stored state. A register is a group of state elements, commonly D flip-flops, retaining a multi-bit value.

Latches

A D latch is transparent while its enable is active: Q follows D during that interval. When enable turns off, it retains the last D. An SR latch has a prohibited/undefined simultaneous assertion case for the standard active-high NOR implementation.

Flip-flops and enable/reset

An edge-triggered D flip-flop samples D at its specified clock edge and otherwise retains Q. An enabled flip-flop changes only when enabled at the active edge. Resettable designs may have synchronous or asynchronous reset; inspect the reset symbol.

D flip-flop at active edge: Q_next = D | otherwise Q retains previous value

EXAM-READY DISTINCTION

A latch is level-sensitive; a flip-flop is edge-sensitive. A changing D does not by itself update a D flip-flop.

06 Latches, Flip-Flops, and Registers

Worked example and independent test

Slides Ch 3: 3-28 | Harris & Harris, pp. 109-122

PROBLEM AND ASSUMPTIONS

Original waveform trace: Q initially 0. D=1 before a rising edge, D switches to 0 after it, then D=0 at the next edge.

Solution steps

1. For the rising-edge D flip-flop, the first edge samples D=1, changing Q from 0 to **1**.
2. The later D change to 0 between edges does not change Q.
3. At the second rising edge Q becomes **0**. For a transparent-high D latch, Q would follow D while enable=1 instead.

Moment	D	DFF Q	Latch Q if enable=1
Before first edge	1	0	1
At first rising edge	1	1	1
Between edges, D changes	0	1	0
At second rising edge	0	0	0

WHAT THIS EXAMPLE PROVES

In a waveform question, identify the device first (latch/edge-triggered FF), then the relevant clock timing.

Try independently

With a falling-edge D flip-flop, should Q change at a rising edge? What if D changes while the clock is high?

07 Finite-State Machines

Moore/Mealy outputs and transition tables

Slides Ch 3: 29-53 | Harris & Harris, pp. 123-140

Basic FSM design

An FSM consists of state memory, next-state logic, and output logic. Begin with desired behavior, choose states, draw transitions, form a state-transition table, assign encodings, derive Boolean equations, and implement registers plus logic.

Moore versus Mealy

A Moore output depends only on current state. A Mealy output depends on current state and current input. Mealy outputs can react within the present cycle, while Moore output changes typically coincide with state changes.

Sequence detectors

States record the meaningful recent history, not arbitrary time counts. For overlapping detections, the next state may retain partial progress. Handle unused state encodings according to whether reset/recovery or don't-cares are specified.

Moore: Y = g(state) | Mealy: Y = g(state,input) | next = f(state,input)

EXAM-READY DISTINCTION

A transition label input/output is Mealy style; a fixed output attached to a state is Moore style.

07 Finite-State Machines

Worked example and independent test

Slides Ch 3: 29-53 | Harris & Harris, pp. 123-140

PROBLEM AND ASSUMPTIONS

Original: build a Mealy detector for three consecutive 1 bits. Use S0=none, S1=one 1, S2=two or more 1s.



Each 1 advances; input 0 returns to S0. At S2, another 1 asserts output.

Solution steps

1. A 0 from any state clears the streak and returns to S0 with output 0.
2. A 1 moves S0 to S1, S1 to S2; from S2 it stays in S2 and outputs 1 on each additional 1.
3. There are three states rather than four because output can depend on the current input in a Mealy design.

Current state	Input 0: next/output	Input 1: next/output
S0	S0 / 0	S1 / 0
S1	S0 / 0	S2 / 0
S2	S0 / 0	S2 / 1

WHAT THIS EXAMPLE PROVES

A Mealy detector can assert output on the third 1 while still using a state that records only the first two 1s.

Try independently

For input 11011110 from S0, identify the cycles on which the detector outputs 1.

08 Sequential Timing and Parallelism

Setup/hold, clock period, latency, throughput

Slides Ch 3: 54-65 | Harris & Harris, pp. 141-161

Timing constraints

Setup time requires D to be stable before the capturing edge; hold time requires it to remain stable after. A path between registers has clock-to-Q delay, combinational delay, and destination setup constraint. Shortest paths govern hold constraints.

Metastability and skew

Violating setup/hold can cause metastability; synchronizers reduce the probability of its propagation. Clock skew changes the effective timing window and can help or harm a particular path depending on the capture clock position.

Temporal and spatial parallelism

Latency measures time from one input to its output. Throughput measures completed results per unit time. Pipelining overlaps steps and can improve throughput without reducing the latency of an individual item. Spatial parallelism duplicates hardware to process multiple items concurrently.

T_clock >= t_clk-to-Q + t_pd(combinational) + t_setup (ignoring skew)

EXAM-READY DISTINCTION

High throughput does not imply short per-item latency. Setup uses maximum path delay; hold uses minimum path delay.

08 Sequential Timing and Parallelism

Worked example and independent test

Slides Ch 3: 54-65 | Harris & Harris, pp. 141-161

PROBLEM AND ASSUMPTIONS

Original: t_clk-to-Q=2 ns, combinational t_pd=7 ns, destination setup=1 ns. Compute minimum clock period and max frequency.

Solution steps

1. The register-to-register critical path needs 2+7+1 = **10 ns** between active edges.
2. Minimum allowed clock period is therefore 10 ns; maximum frequency is 1/(10 ns) = **100 MHz**.
3. If the combinational minimum delay is 0.5 ns, a separate hold-time check is still necessary; the maximum-delay calculation cannot certify it.

Metric	Meaning	Typical objective
Latency	Time for one item to complete	Minimize
Throughput	Items finished per second	Maximize
Setup constraint	Data ready before sampling edge	Check longest path
Hold constraint	Data remains after sampling edge	Check shortest path

WHAT THIS EXAMPLE PROVES

Do not use a propagation-delay maximum to prove a hold-time minimum.

Try independently

If a 5-stage pipeline completes one item each 2 ns after filling, what is steady-state throughput?

09 Adders, ALUs, and Carry Delay

From 1-bit addition to N-bit arithmetic circuits

Slides Ch 5: 4-29 | Harris & Harris, pp. 239-254

Full adder

A 1-bit full adder adds A, B, and C_in. Its sum is A XOR B XOR C_in; carry-out is AB + AC_in + BC_in. An N-bit ripple-carry adder chains carry outputs to the next bit, producing a linear worst-case carry delay.

Faster carry and subtraction

Carry-lookahead calculates generate/propagate information so upper carries need not ripple through each bit. Two's-complement subtraction can be implemented as A + (~B) + 1. Comparators and an ALU select arithmetic or logic functions using control inputs.

Shifts and overflow

Logical shifts fill vacated bits with zero; arithmetic right shift preserves a sign bit. Fixed-width arithmetic can overflow. Addition overflow depends on signed versus unsigned interpretation.

Sum = A xor B xor C_in | C_out = AB + AC_in + BC_in

EXAM-READY DISTINCTION

A ripple-carry circuit is limited by the longest carry dependency, not by the number of sum bits alone.

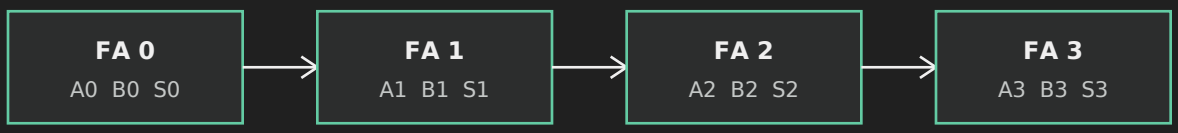
09 Adders, ALUs, and Carry Delay

Worked example and independent test

Slides Ch 5: 4-29 | Harris & Harris, pp. 239-254

PROBLEM AND ASSUMPTIONS

Original: add 0101 and 0111 with a 4-bit ripple-carry adder starting C_in=0.



Carry-out of each full adder becomes next stage carry-in.

Solution steps

1. Bit 0: 1+1+0 gives sum 0 and carry 1.
2. Bit 1: 0+1+1 gives sum 0 and carry 1. Bit 2: 1+1+1 gives sum 1 and carry 1.
3. Bit 3: 0+0+1 gives sum 1 and carry 0. Final sum is **1100**, decimal 12; no unsigned carry-out.

Bit i	A_i	B_i	Carry in	Sum	Carry out
0	1	1	0	0	1
1	0	1	1	0	1
2	1	1	1	1	1
3	0	0	1	1	0

WHAT THIS EXAMPLE PROVES

With 4-bit signed values, 0101 (+5) + 0111 (+7) yields 1100 (-4), so signed overflow occurred.

Try independently

For a ripple-carry chain of 16 stages with 450 ps worst carry delay per stage, compute the simple worst-case carry-path estimate.

10 Fixed Point and IEEE 754 Floating Point

Sign, biased exponent, fraction, and normalization

Slides Ch 5: 30-46 | Harris & Harris, pp. 255-260

Fixed point

In fixed-point representation, a predetermined number of bits are fractional. Their weights are 1/2, 1/4, 1/8, etc. Changing scale affects range and precision.

Single-precision fields

IEEE 754 single precision uses a 1-bit sign, 8-bit biased exponent (bias 127), and 23-bit fraction field. For normalized nonzero values, the significand is 1.fraction in binary; the leading 1 is implicit. Special all-zero and all-one exponent values require separate interpretation.

Decode method

Convert hex into 32 binary bits; split 1|8|23; determine sign, subtract 127 from encoded exponent, then calculate (1+fraction)*2^exponent. Do not treat exponent bits as an ordinary signed two's-complement value.

Normalized value = (-1)^sign * (1 + fraction) * 2^(stored_exponent - 127)

EXAM-READY DISTINCTION

The exponent field is biased, not two's complement. Exponent 255 designates Inf/NaN, not an ordinary finite value.

10 Fixed Point and IEEE 754 Floating Point

Worked example and independent test

Slides Ch 5: 30-46 | Harris & Harris, pp. 255-260

PROBLEM AND ASSUMPTIONS

Slide/sample-exam style: decode 0xC0B00000 as IEEE 754 single precision.

Solution steps

1. Hex to bit fields: 1 | 10000001 | 01100000000000000000000.
2. Sign=1, so negative. Encoded exponent 10000001 binary =129, and 129-127 = 2.
3. Fraction bits 011... contribute 1/4 + 1/8 = 0.375; significand = 1.375. Thus value is -(1.375)(2^2) = **-5.5**.

Field	Bit pattern	Interpretation
Sign (1)	1	negative
Exponent (8)	10000001	129; unbiased 2
Fraction (23)	01100000000000000000000	0.375
Value	-1.375 * 4	-5.5

WHAT THIS EXAMPLE PROVES

Split fields before calculating. A hex prefix of C0 does not by itself tell the whole floating-point value.

Try independently

Encode +2.625 in IEEE 754 single precision. First normalize 10.101 binary as 1.0101 * 2^1.

11 Counters, Registers, and Memory Arrays

Sequential blocks and address-decoded storage

Slides Ch 5: 47-70 | Harris & Harris, pp. 260-280

Registers and counters

An N-bit register holds N bits. Shift registers move bits on clock events and may support parallel load. A modulo-2^N binary counter wraps to zero when its N-bit count overflows.

Memory organizations

A memory array of 2^A words needs A address bits; each word contains W data bits. Thus capacity is 2^A * W bits. Read and write ports, enable signals, and decoders determine how locations are selected.

ROM, RAM, PLA

SRAM and DRAM are volatile read/write memories with different cell designs. ROM stores fixed or programmed patterns; a ROM can implement truth tables. Programmable logic arrays (PLA) realize SOP logic through configurable AND and OR planes.

Capacity = number_of_words * bits_per_word | address bits = log2(number_of_words)

EXAM-READY DISTINCTION

A 256 x 8 memory has 256 addressable words of 8 bits each; 256 is not the data-word width.

11 Counters, Registers, and Memory Arrays

Worked example and independent test

Slides Ch 5: 47-70 | Harris & Harris, pp. 260-280

PROBLEM AND ASSUMPTIONS

Original: a memory has 1,024 words, 16 bits per word. Find address bits, total storage, and output bus width.

Solution steps

1. Because 1,024=2^10, it requires **10 address bits** to select one word.
2. Storage is 1,024 * 16 = **16,384 bits** or 2,048 bytes (2 KiB).
3. Its data bus is **16 bits** wide for a complete word access; address and data buses describe different dimensions.

Property	Value	Reason
Address bits	10	1024 distinct words
Word width	16	16 bits per access
Capacity	16,384 bits	1024 * 16
Capacity bytes	2,048 bytes	divide bits by 8

WHAT THIS EXAMPLE PROVES

A decoder selects a word; a word-sized data bus carries that word's contents.

Try independently

For a 64 x 32 ROM, how many address bits and total bytes are present?

12 MIPS Registers, Operands, and Memory

32 registers, lw/sw addressing, and immediates

Slides Ch 6: 2-21, 24 | Harris & Harris, pp. 295-305

Assembly and register conventions

MIPS uses 32 general-purpose 32-bit registers. $zero always reads zero; $t0-$t9 are temporary registers, $s0-$s7 are saved registers, $a0-$a3 are arguments, $v0-$v1 are return values, $sp points to the stack, and $ra stores a return address.

Memory and addressing

MIPS arithmetic generally operates on registers, not memory locations. Use lw (load word) and sw (store word) to transfer 32-bit words. In byte-addressable memory, successive aligned words begin 4 bytes apart. Offset(base) computes an effective byte address by adding a signed offset to the base register.

Immediates

addi supplies a 16-bit signed constant as the third operand. Destination comes first: add rd,rs,rt; addi rt,rs,imm. Typical lw form: lw rt,offset(rs). Be careful: sw writes memory; it does not modify the register being stored.

Effective address = value(base register) + signed offset in bytes

---

EXAM-READY DISTINCTION

The registers are not memory addresses by default. The value inside a base register contributes to the byte address.

12 MIPS Registers, Operands, and Memory

Worked example and independent test

Slides Ch 6: 2-21, 24 | Harris & Harris, pp. 295-305

PROBLEM AND ASSUMPTIONS

Original: $s1=0x10010020, $t0=27, and memory[0x10010028]=99. Trace lw $t1,8($s1), then sw $t0,12($s1).

Solution steps

1. Load effective address = 0x10010020 + 8 = **0x10010028**; therefore $t1 becomes 99.
2. Store effective address = 0x10010020 + 12 = **0x1001002C**; memory at that address becomes 27.
3. Neither operation changes $s1, and the store leaves $t0=27. Check that word accesses are 4-byte aligned.

Register(s)	Typical purpose
$zero / $0	Hardwired zero
$a0-$a3, $v0-$v1	Call arguments / return values
$t0-$t9	Caller-saved temporaries
$s0-$s7	Callee-saved variables
$sp, $ra	Stack pointer / return address

WHAT THIS EXAMPLE PROVES

lw and sw use effective byte addresses. A 32-bit word occupies four byte addresses.

Try independently

If $s0=0x10010040, at what address does lw $t2,-16($s0) read?

13 Byte Ordering and Signed Immediate Values

Endian conventions and address interpretation

Slides Ch 6: 17-24 | Harris & Harris, pp. 300-305

Byte-addressable memory

A 32-bit word comprises four consecutive bytes. A load or store word uses its starting byte address, which should be a multiple of four in the introductory MIPS model. lw and sw move whole words; lb/sb handle bytes.

Endianness

Big-endian stores the most significant byte of a multibyte word at the lowest address. Little-endian stores the least significant byte there. If all bytes of a word are retrieved and interpreted consistently, word value remains unchanged.

Signed versus unsigned byte loads

A signed byte load sign-extends the highest byte bit; an unsigned byte load zero-extends it. The slide example with 0x23456789 has nonnegative constituent bytes, so both interpretations happen to match for these particular bytes.

Immediates and offsets

A signed 16-bit immediate is sign-extended before relevant 32-bit arithmetic. An immediate offset of -4 refers to four bytes before the register address, not four words earlier.

Big endian: byte at lowest address = MSB | Little endian: byte at lowest address = LSB

EXAM-READY DISTINCTION

Endian diagrams refer to increasing byte addresses; do not reverse a hex number's printed digit order blindly.

13 Byte Ordering and Signed Immediate Values

Worked example and independent test

Slides Ch 6: 17-24 | Harris & Harris, pp. 300-305

PROBLEM AND ASSUMPTIONS

Slide-derived example: $t0=0x23456789; execute sw $t0,0($zero), then lb $s0,1($zero).

Big-endian

23	45	67	89
addr 0	addr 1	addr 2	addr 3

Little-endian

89	67	45	23
addr 0	addr 1	addr 2	addr 3

Solution steps

1. Big-endian layout: address 0:23, 1:45, 2:67, 3:89 (hex bytes). Loading byte 1 gives 0x45, or **69**.
2. Little-endian layout: address 0:89, 1:67, 2:45, 3:23. Loading byte 1 gives 0x67, or **103**.
3. Both bytes have top bit 0, so sign extension yields 0x00000045 and 0x00000067 respectively.

Byte address	0	1	2	3
Big endian	23	45	67	89
Little endian	89	67	45	23

WHAT THIS EXAMPLE PROVES

The question uses lb at byte offset one, so the answer differs even though the stored 32-bit word is the same.

Try independently

Repeat with lw $s0,0($zero). Would $s0 differ between endian systems if the processor uses its own convention consistently?

14 R-Type, I-Type, and J-Type Encoding

The 32-bit instruction fields and opcode rules

Slides Ch 6: 25-34 | Harris & Harris, pp. 305-310

R-type instructions

R-type layout: op(6), rs(5), rt(5), rd(5), shamt(5), funct(6). For the introductory arithmetic R-type operations, opcode 0 and the funct field identifies the operation. Read assembly as operation destination, source 1, source 2.

I-type instructions

I-type layout: op(6), rs(5), rt(5), immediate(16). It is used by addi, lw, sw, beq, bne and more. In arithmetic immediates, rt commonly receives the result. For sw, rt contains the data to store; for branches, both rs and rt are compared.

J-type instructions

J-type layout: op(6), address field(26). Its field does not store an arbitrary 32-bit byte address. Jump reconstruction appends two zero bits and uses high bits from PC+4 in the classic MIPS scheme.

Decoding

First examine op. For op=0, interpret the R-type fields and funct. For nonzero op, use the opcode to determine instruction type and meaning. Register names map to fixed register numbers.

R: 6 | 5 | 5 | 5 | 5 | 6 I: 6 | 5 | 5 | 16 J: 6 | 26

EXAM-READY DISTINCTION

Do not read an I-type rt field as a destination for every instruction. Its role depends on the opcode.

14 R-Type, I-Type, and J-Type Encoding

Worked example and independent test

Slides Ch 6: 25-34 | Harris & Harris, pp. 305-310

PROBLEM AND ASSUMPTIONS

Lecture-derived R-type example: encode add $s0,$s1,$s2.

op	rs	rt	rd	shamt	funct
000000	10001	10010	10000	00000	100000
6 bits	5 bits	5 bits	5 bits	5 bits	6 bits

Solution steps

1. Registers: $s1=17, $s2=18, and $s0=16. The add funct code is 32 (binary 100000).
2. Fields: op=0; rs=17; rt=18; rd=16; shamt=0; funct=32.
3. Concatenate 000000 10001 10010 10000 00000 100000, giving **0x02328020**.

Field	op	rs	rt	rd	shamt	funct
Width	6	5	5	5	5	6
Value	0	17	18	16	0	32

WHAT THIS EXAMPLE PROVES

For add, the first register written in assembly is rd, but the binary field order is rs, rt, rd.

Try independently

Decode an R-type instruction with rs=11, rt=13, rd=8, shamt=0, funct=34. Identify operation and register names.

15 MIPS Logic, Shifts, and Constants

Bitwise instructions and control flow

Slides Ch 6: 35-48 | Harris & Harris, pp. 310-316

Bitwise operations

and/or/xor/nor operate independently on corresponding bit positions. AND with a mask clears selected bits; OR combines set bits. To invert 32 bits, NOR with $zero computes bitwise NOT.

Shifts

sll shifts left, filling lower bits with zeros. srl shifts right, filling upper bits with zeros. Shift amounts are encoded in the shamt field for immediate-shift R-type instructions; variable shifts read an amount from a register.

Branching

beq rs,rt,label branches when equal; bne branches when unequal. j is unconditional, jr targets an address in a register, and jal supports function calls. The PC normally advances to the next instruction; branches modify the flow.

Immediates and constants

addi uses a signed 16-bit immediate. Building a larger 32-bit constant may require lui to set upper bits and ori to fill lower bits; details depend on whether the instruction is genuine or a pseudoinstruction.

---

sll: bits << shamt | beq: branch when rs = rt | bne: branch when rs != rt

---

EXAM-READY DISTINCTION

Logical shift-right fills with zeros; it is not equivalent to arithmetic-right shift on negative signed values.

15 MIPS Logic, Shifts, and Constants

Worked example and independent test

Slides Ch 6: 35-48 | Harris & Harris, pp. 310-316

PROBLEM AND ASSUMPTIONS

Original: $t0=0xF234012F. Calculate andi $t1,$t0,0x00FF, then sll $t2,$t1,2.

```
andi $t1, $t0, 0x00FF   # keep low byte
sll  $t2, $t1, 2       # shift two places
```

Solution steps

1. ANDing with immediate 0x00FF preserves the low eight bits only. 0xF234012F AND 0x000000FF = **0x0000002F**.
2. Logical left shift by two gives 0x2F * 4 = 0xBC. Thus $t2 = **0x000000BC**.
3. Neither operation changes $t0. The shift does not rotate the discarded high-order bits back around.

Operation	Value (hex)
Start $t0	F234012F
Mask	000000FF
AND result $t1	0000002F
Shift result $t2	000000BC

WHAT THIS EXAMPLE PROVES

Each 1 in the mask selects a bit to retain. A shift amount of two multiplies an unsigned value by four if no 1 bits are lost.

Try independently

What does nor $t3,$t0,$zero compute? Why is it a bitwise complement?

16 C Conditionals and Loops in MIPS

Branch labels, slt, and program tracing

Slides Ch 6: 49-55 | Harris & Harris, pp. 314-320

If and if/else

A branch either skips a block or selects an alternate block. Unlike high-level braces, labels mark instruction addresses. Translate one condition at a time; a common pattern branches around the THEN body when the condition is false.

Comparisons

slt rd,rs,rt writes 1 when rs<rt, else 0 (signed form). Other comparisons can combine slt, beq/bne, and $zero. Watch strict inequality versus <=; off-by-one branches are common.

Loops

A while loop normally tests at the top and exits when false; a for loop adds initialization and a loop update. Tracing a loop requires a register-state table and a clear test point. Keep track of whether increments occur before or after the body.

Branch offsets

beq/bne use a PC-relative offset in words from PC+4. The 16-bit immediate is sign-extended and shifted left by two in the classic instruction-address computation.

Branch target = (PC + 4) + (sign_extended_imm << 2)

EXAM-READY DISTINCTION

A branch label names an address; its encoding uses a relative word offset, not its literal byte address.

16 C Conditionals and Loops in MIPS

Worked example and independent test

Slides Ch 6: 49-55 | Harris & Harris, pp. 314-320

PROBLEM AND ASSUMPTIONS

Original: translate while(i<4) { i=i+1; } using $s0=i (initialized elsewhere).

```
LOOP: slti $t0, $s0, 4
      beq  $t0, $zero, DONE
      addi $s0, $s0, 1
      j    LOOP
DONE: # continue
```

Solution steps

1. At LOOP, compute whether i<4 using slti $t0,$s0,4. This writes 1 only while the loop should execute.
2. Branch out when $t0=0; otherwise increment i and jump back to LOOP.
3. If i starts at 1, bodies execute at i=1,2,3; the loop ends with **i=4**, after three increments.

Test at LOOP	slt result	Action
i=1	1	increment to 2
i=2	1	increment to 3
i=3	1	increment to 4
i=4	0	branch to DONE

WHAT THIS EXAMPLE PROVES

Observe the final failing test: when i=4, the branch exits without another increment.

Try independently

Modify the test to express while(i<=4). How many iterations occur if i starts at 1?

17 MIPS Arrays and Index Arithmetic

Compute element addresses correctly

Slides Ch 6: 56-60 | Harris & Harris, pp. 320-325

Array address calculation

An array base address identifies element zero. For a 32-bit integer array, A[i] lives at base + 4*i because each int occupies four bytes. Multiply the index by four using sll by two (or appropriate arithmetic).

Read and write

lw loads the element from computed address into a register. sw stores a register value into it. For byte arrays, the scale factor is one; for double-word arrays, it may be eight. Always use the element size specified by the problem.

Array loops

In MIPS, the loop commonly maintains an index register, tests a bound, computes address, accesses memory, updates the index, and branches back. Distinguish element number from its byte offset.

Correctness

Verify the base register is not accidentally overwritten and that the calculated address is aligned for word access. When an array is local to a function, its base can be in a saved register or on the stack according to code conventions.

Address(A[i]) = base(A) + i * element_size_in_bytes

EXAM-READY DISTINCTION

Do not use byte offset i for an integer array element i. The index must be scaled by four.

17 MIPS Arrays and Index Arithmetic

Worked example and independent test

Slides Ch 6: 56-60 | Harris & Harris, pp. 320-325

PROBLEM AND ASSUMPTIONS

Original: $s0 contains base address 0x10010000 and $s1=3. Find A[3] and store 42.

```
sll  $t0, $s1, 2      # i * 4
add  $t1, $s0, $t0    # address A[i]
addi $t2, $zero, 42
sw   $t2, 0($t1)
```

Solution steps

1. Compute byte offset = 3 * 4 = 12 decimal = 0xC. Address of A[3] is **0x1001000C**.
2. sll multiplies index 3 by 4, producing 12. Add the base address to obtain a pointer.
3. Put 42 in $t2 and sw it through the computed pointer. The index $s1 remains 3.

Element	Byte offset	Address
A[0]	0	0x10010000
A[1]	4	0x10010004
A[2]	8	0x10010008
A[3]	12	0x1001000C

WHAT THIS EXAMPLE PROVES

The shift does not change the array value; it only computes the memory location of the selected element.

Try independently

For A[7] of 32-bit words at 0x10010080, what is the byte offset and address?

18 Functions, Stack, and Recursion

Caller/callee conventions and saved state

Slides Ch 6: 63-83 | Harris & Harris, pp. 325-333

Call and return

jal label saves return address in $ra and transfers control to the function. jr $ra returns to the caller. The first four arguments typically arrive in $a0-$a3; results are returned in $v0-$v1.

Register preservation

Saved $s-registers must be restored by the callee if it changes them. Temporary $t-registers and argument registers are caller-saved: the caller must preserve a live value before calling a function. A nested function call overwrites $ra unless the caller-function saves it.

Stack discipline

The MIPS stack grows toward lower addresses. Allocate a frame by subtracting from $sp; use sw to save needed values, and lw to restore. Deallocate by adding the same amount back to $sp. Balanced prologue/epilogue preserve stack integrity.

Recursion

Each recursive invocation must maintain its own independent saved registers, return address, and local values. Frames stack up while descending and unwind in reverse order on return.

Allocate: addi $sp,$sp,-N | Release: addi $sp,$sp,+N

EXAM-READY DISTINCTION

Do not treat $ra as automatically safe across nested jal. Recursion generally needs a saved return address in each active frame.

18 Functions, Stack, and Recursion

Worked example and independent test

Slides Ch 6: 63-83 | Harris & Harris, pp. 325-333

PROBLEM AND ASSUMPTIONS

Original: function f changes $s0 and calls another function. Show a minimal illustrative save/restore for $s0 and $ra.

```
f:    addi $sp, $sp, -8
      sw   $s0, 0($sp)
      sw   $ra, 4($sp)
      # body, possibly jal g
      lw   $s0, 0($sp)
      lw   $ra, 4($sp)
      addi $sp, $sp, 8
      jr   $ra
```

Solution steps

1. Reserve 8 bytes; save incoming $s0 and $ra in two 4-byte stack slots.
2. The function body may change $s0 and perform jal g, which changes $ra. The saved return address is still protected on the stack.
3. Restore both registers before releasing the frame and returning. Afterward $sp is exactly its original value.

New frame slot	Saved value	Why
0($sp)	$s0	Callee must preserve
4($sp)	$ra	Nested jal would overwrite
After epilogue	Original $sp	Frame removed completely

WHAT THIS EXAMPLE PROVES

The order of saved slots is a local convention; consistent offsets and correct restoration are what matter.

Try independently

What happens if f performs jal g without saving its own return address and then tries jr $ra?

19 Addressing Modes and Stored Programs

How instructions find data and next PCs

Slides Ch 6: 84-102 | Harris & Harris, pp. 333-345

MIPS addressing modes

Register addressing obtains operands directly from registers. Base addressing adds a signed immediate offset to a base register (lw/sw). PC-relative addressing is used for conditional branches. Pseudodirect addressing combines the jump target field with upper PC bits.

Stored-program concept

Instructions and data are both bit patterns in memory. The program counter identifies the next instruction to fetch; sequential 32-bit MIPS instructions advance PC by four bytes unless control flow redirects it.

Translation and execution

Compiling generates assembly, assembling converts mnemonics and labels into machine code, linking resolves symbol references, and loading places executable content in memory. The course slides show text, data, heap, and stack regions in the MIPS memory map.

Exceptions and pseudoinstructions

Pseudoinstructions expand into real MIPS instructions. Exceptions transfer control to an exception handler, often recording a cause and EPC. These are identified in the final Ch 6 lecture slides, but are lower priority than the eLearning objectives.

Sequential next PC = PC + 4; branch target = PC + 4 + 4 * signed_offset

EXAM-READY DISTINCTION

Pseudoinstructions are an assembly-language convenience; do not assume they correspond to one machine instruction.

19 Addressing Modes and Stored Programs

Worked example and independent test

Slides Ch 6: 84-102 | Harris & Harris, pp. 333-345

PROBLEM AND ASSUMPTIONS

Original: beq at PC=0x00400020 with immediate +3, taken. Compute its target.

Solution steps

1. The address of the next sequential instruction is PC+4 = **0x00400024**.
2. The signed branch offset is measured in words: 3 * 4 = 12 bytes = 0xC.
3. Taken target = 0x00400024 + 0xC = **0x00400030**. If the branch is not taken, next PC remains 0x00400024.

Address mode	Example	How location is computed
Register	add $t0,$s1,$s2	Register numbers
Base	lw $t0,8($s1)	value($s1)+8
PC-relative	beq $t0,$t1,L	PC+4+4*offset
Pseudodirect	j TARGET	upper PC bits + field<<2

WHAT THIS EXAMPLE PROVES

Conditional branch immediate counts four-byte instructions, not literal bytes from the current PC.

Try independently

Repeat with offset -2 and PC 0x00400020. Confirm the target is before the next sequential instruction.

20 Microarchitecture and Performance Measures

Compare processor organization using execution time

Slides Ch 7: 3-8 | Harris & Harris, pp. 371-376

Architecture versus microarchitecture

The instruction set architecture (ISA) specifies the visible programmer-level machine (registers, instructions, defined effects). Microarchitecture is how a processor implements those effects using hardware datapaths, control, and state elements. One ISA can have many implementations.

Three designs

Single-cycle executes a complete instruction in one long clock cycle (CPI=1). Multicycle reuses hardware over several shorter clock cycles, with instruction-dependent CPI. Pipelining overlaps different instructions in multiple stages, improving steady-state throughput.

Performance math

Instruction execution time depends on instruction count, average cycles per instruction, and clock period. Clock frequency is the reciprocal of clock period. Lower CPI alone is not proof of better performance if the period differs.

Latency versus throughput

Latency is the completion time for one instruction; throughput is instructions completed per unit time. Deep pipelining can improve throughput while leaving one instruction's latency relatively high.

CPU time = instruction_count * average_CPI * clock_period

EXAM-READY DISTINCTION

Compare total time rather than only frequency or CPI. Multiply CPI by the cycle time before judging a design.

20 Microarchitecture and Performance Measures

Worked example and independent test

Slides Ch 7: 3-8 | Harris & Harris, pp. 371-376

PROBLEM AND ASSUMPTIONS

Original: processor A has CPI 1 and clock 12 ns; processor B has average CPI 4.0 and clock 2.5 ns. Both execute 1 million instructions.

Solution steps

1. A: 1,000,000 * 1 * 12 ns = **12 ms**.
2. B: 1,000,000 * 4 * 2.5 ns = **10 ms**.
3. Despite using four times as many cycles per instruction, B executes this workload faster because its clock period is much shorter.

Design	CPI	Period	CPU time, N=10^6
A single-cycle	1.0	12 ns	12 ms
B multicycle	4.0	2.5 ns	10 ms

WHAT THIS EXAMPLE PROVES

Different processor designs can run identical MIPS instructions but need not have the same CPI or clock speed.

Try independently

With 500,000 instructions, CPI 1.5, and 3 ns cycles, compute execution time and instructions/second.

21 Single-Cycle Datapath and Control

Trace lw, sw, and R-type instructions

Slides Ch 7: 9-17 | Harris & Harris, pp. 376-388

Major hardware blocks

A PC addresses instruction memory; the register file supplies source registers; the ALU adds, compares, or performs arithmetic; data memory serves loads/stores; multiplexers select destinations, operands, write-back data, and the next PC.

Load word path

For lw: fetch instruction, read base register, sign-extend the immediate, ALU computes effective address, data memory reads a word, and register file writes that value to rt. PC normally advances by four.

Store and R-type

For sw: read base and value register, ALU computes effective address, and data memory writes; no register is written. For add: read rs/rt, ALU adds, and write result to rd; data memory is not used for the instruction.

Control unit

Instruction opcode and sometimes funct determine control lines such as RegWrite, MemWrite, RegDst, ALUSrc, MemtoReg, Branch, and ALUControl. Signals enable the relevant hardware without changing the ISA.

---

lw: register -> ALU address -> memory -> register | sw: register -> ALU -> memory write

---

EXAM-READY DISTINCTION

A store must assert MemWrite, not RegWrite. An R-type add writes rd; an lw writes rt.

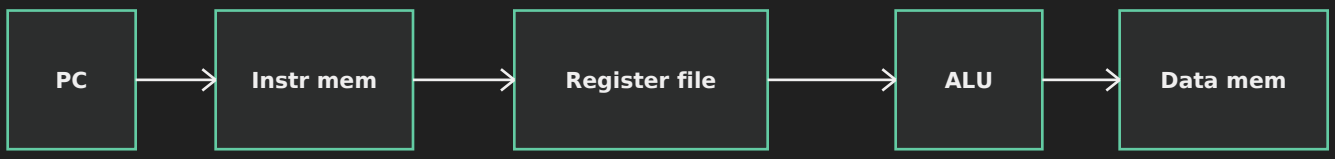
21 Single-Cycle Datapath and Control

Worked example and independent test

Slides Ch 7: 9-17 | Harris & Harris, pp. 376-388

PROBLEM AND ASSUMPTIONS

Original: classify which hardware blocks are used by lw $t0,12($s1), sw $t2,8($s1), and add $s0,$t0,$t2.



Simplified data flow for lw; control and return-to-register paths omitted for clarity

Solution steps

1. lw uses the immediate as ALU operand, reads data memory, and writes the memory result into $t0.
2. sw similarly uses an immediate offset to address memory but sends the second register's value to memory and disables register write.
3. add uses register operands through the ALU, selects rd=$s0 as the destination, and writes the ALU result. It does not assert MemWrite.

Signal	lw	sw	R-type add
RegWrite	1	0	1
MemWrite	0	1	0
ALUSrc	1	1	0
RegDst	0	X	1
MemtoReg	1	X	0

WHAT THIS EXAMPLE PROVES

X means don't-care when the corresponding operation is disabled; it is not a demand for a particular 0/1.

Try independently

For add $s0,$s1,$s2, which register number belongs in the write-register address and what write-back source is selected?

22 Single-Cycle Critical-Path Timing

Why a load sets the clock period

Slides Ch 7: 17-19 | Harris & Harris, pp. 388-389

One instruction per cycle

In a single-cycle processor every implemented instruction starts and completes between consecutive edges. Cycle time must be long enough for the slowest instruction through its required combinational blocks.

Load critical path

The lecture's single-cycle timing exercise assigns 4 ns to instruction/data memory, 3 ns to the ALU and adders, and 1 ns for register read or write. It explicitly ignores mux, control, sign extension, PC access, and shift delays.

Instruction paths

A load requires instruction memory, register read, address ALU, data-memory read, and register write. An R-type ALU instruction skips data-memory read; sw does not include register write-back.

Consequence

All instructions, even faster ones, consume the full single-cycle period because the clock cannot end earlier for just one type. This simplicity motivates multicycle and pipelined designs.

---

t_clock >= max(all instruction data-path delays) | CPI(single-cycle) = 1

EXAM-READY DISTINCTION

Use precisely the delay assumptions in the problem. Do not add unlisted mux/control delays when explicitly told to ignore them.

22 Single-Cycle Critical-Path Timing

Worked example and independent test

Slides Ch 7: 17-19 | Harris & Harris, pp. 388-389

PROBLEM AND ASSUMPTIONS

Lecture-based: instruction/data memory 4 ns each, ALU 3 ns, register read/write 1 ns each. Find slowest single-cycle path.

Solution steps

1. Load word: instruction fetch 4 + register read 1 + address ALU 3 + data-memory read 4 + register write 1 = **13 ns**.
2. R-type add: instruction fetch 4 + register read 1 + ALU 3 + register write 1 = **9 ns**.
3. Store: instruction fetch 4 + register read 1 + address ALU 3 + memory write 4 = **12 ns**. So load limits the clock to 13 ns in this simplified set.

Instruction	Path components (ns)	Total
lw	4 + 1 + 3 + 4 + 1	13 ns
sw	4 + 1 + 3 + 4	12 ns
R-type add	4 + 1 + 3 + 1	9 ns

WHAT THIS EXAMPLE PROVES

Once the slowest path is known, every instruction consumes the same clock period in the single-cycle design.

Try independently

What is the maximum clock frequency for the 13 ns cycle? Why is 1/9 ns not the single-cycle CPU frequency?

23 Multicycle Datapath and CPI

Resource reuse, staged control, instruction mixtures

Slides Ch 7: 19-29 | Harris & Harris, pp. 389-409

Multicycle design

Break instruction execution into shorter operations such as fetch, register read/decode, execute/address calculation, memory access, and write-back. Intermediate registers (e.g., instruction/data registers and ALU output storage) retain results between cycles.

Instruction-dependent cycles

The lecture says beq and j take 3 cycles; sw, addi, and R-type take 4; lw takes 5 in its example design. The average CPI is the weighted mean across instruction classes.

Timing tradeoff

Because each stage performs less combinational work than a full instruction, multicycle clock period can be shorter. Control is more complex and some instructions use more cycles, so performance depends on the instruction mix and clock time.

Cycle time

The multicycle cycle time must accommodate the slowest individual micro-operation, including the appropriate sequential overhead when stated. It is not the sum of all stages for lw.

Average CPI = sum over classes (instruction_fraction * cycles_per_instruction)

EXAM-READY DISTINCTION

Fractions must add to 100%. A 5-cycle load is not the average CPI of the workload.

23 Multicycle Datapath and CPI

Worked example and independent test

Slides Ch 7: 19-29 | Harris & Harris, pp. 389-409

PROBLEM AND ASSUMPTIONS

Lecture-derived: 25% lw; 10% sw; 11% branches; 2% jumps; 52% R-type. Compute average CPI.

Solution steps

1. Group the 3-cycle classes: (0.11+0.02)*3 = 0.39.
2. Group the 4-cycle classes: (0.10+0.52)*4 = 2.48.
3. Loads contribute 0.25*5 = 1.25. Sum 0.39+2.48+1.25 = **4.12 CPI**, matching the lecture slide.

Type	Frequency	Cycles	Weighted CPI
lw	25%	5	1.25
sw	10%	4	0.40
branch	11%	3	0.33
jump	2%	3	0.06
R-type	52%	4	2.08
Total	100%	--	4.12

WHAT THIS EXAMPLE PROVES

The speed comparison requires 4.12 * multicycle clock period versus 1 * single-cycle clock period.

Try independently

If multicycle clock period is 3 ns, calculate average instruction time. Is it faster than a 13 ns single-cycle processor?

24 Pipelining and Stage Timing

Five overlapped stages and throughput

Slides Ch 7: 30-33 | Harris & Harris, pp. 409-414

Five-stage MIPS pipeline

The standard stages are IF (fetch), ID (decode/register read), EX (ALU), MEM (data memory), and WB (register write-back). Pipeline registers isolate each stage and carry data/control into the next cycle.

Overlap principle

Instruction 2 enters IF while instruction 1 is in ID; after filling, ideally one instruction finishes per cycle. Five stages do not mean five instructions finish in every cycle. Startup and drain add cycles to a short instruction sequence.

Timing diagrams

Draw instructions as rows and clock cycles as columns; place IF, ID, EX, MEM, WB on successive diagonals. With no stalls, N instructions take N+4 cycles in a 5-stage pipeline from first IF to last WB.

Performance limitation

The pipelined clock period is set by the slowest stage (plus pipeline-register overhead if specified). Stage imbalance, hazards, and flushes reduce ideal speedup.

Ideal 5-stage pipeline: cycles for N instructions = N + 4 | steady-state CPI ≈ 1

EXAM-READY DISTINCTION

The pipeline increases instruction throughput by overlap, but a single instruction still traverses all five stages.

24 Pipelining and Stage Timing

Worked example and independent test

Slides Ch 7: 30-33 | Harris & Harris, pp. 409-414

PROBLEM AND ASSUMPTIONS

Original: show an ideal five-stage pipeline executing four independent instructions, I1-I4, with no stalls.

Solution steps

1. I1 occupies cycles 1-5; I2 occupies 2-6; I3 occupies 3-7; I4 occupies 4-8.
2. The last WB occurs in cycle 8: N+4=4+4 = **8 cycles**. An unpipelined 5-cycle sequence would take 4*5=20 cycles under equal stage lengths.
3. The improvement comes from overlap, not from deleting any work required by an individual instruction.

Instruction	C1	C2	C3	C4	C5	C6	C7	C8
I1	IF	ID	EX	MEM	WB			
I2		IF	ID	EX	MEM	WB		
I3			IF	ID	EX	MEM	WB	
I4				IF	ID	EX	MEM	WB

WHAT THIS EXAMPLE PROVES

When pipeline diagrams have stalls or branches, the simple N+4 formula no longer applies without adjustment.

Try independently

How many ideal cycles do ten independent instructions need in a five-stage pipeline?

25 RAW Data Hazards, Forwarding, and Stalls

Know when a source operand becomes ready

Slides Ch 7: 34-40 | Harris & Harris, pp. 414-426

Read-after-write dependence

A RAW hazard occurs when a younger instruction reads a register before an earlier instruction has supplied its new value. It is a true data dependence. Identify the producing destination and each consuming source, not just identical register names.

Compiler solutions

A compiler can insert nop instructions or move independent useful instructions between a producer and consumer. Reordering must preserve meaning: moving an instruction that writes a needed register may introduce another hazard.

Forwarding

Bypass networks send an available ALU result directly from a later pipeline register to a consuming EX input without waiting for register-file write-back. Forwarding often resolves back-to-back arithmetic RAW hazards.

Load-use hazard

In the standard five-stage pipeline, a load's memory result arrives later than an ALU result. A consumer immediately after lw still needs a one-cycle stall even with ordinary forwarding, as shown in the lecture's stalling diagram.

RAW: earlier instruction writes register R; later instruction reads that same R

EXAM-READY DISTINCTION

Forwarding is not the same as avoiding a dependence. A load-use delay may require an inserted bubble even with bypassing.

25 RAW Data Hazards, Forwarding, and Stalls

Worked example and independent test

Slides Ch 7: 34-40 | Harris & Harris, pp. 414-426

PROBLEM AND ASSUMPTIONS

Lecture-style original: lw $s0,0($s1) immediately followed by add $t0,$s0,$t2. Determine hazard and remedy.

Solution steps

1. The load produces $s0, and the add consumes $s0: a RAW load-use dependence.
2. In a five-stage textbook pipeline the load value becomes available after MEM, too late for the next instruction's immediately following EX. A **one-cycle stall** separates the consumer's EX from the producer's MEM.
3. The new value can then be forwarded into the add EX stage; other independent instructions could instead be scheduled between the two.

Pair	RAW?	Usual remedy with forwarding
add $s0,...; and $t0,$s0,$s1	Yes	ALU forwarding
lw $s0,0($s1); add $t0,$s0,$t2	Yes	One stall + forwarding
sw $t0,0($s1); add $t2,$s1,$s0	No $t0 use	No related $t0 RAW

WHAT THIS EXAMPLE PROVES

The necessary stall count depends on the pipeline timing assumptions; use the lecture's five-stage model.

Try independently

Does add $t0,$s1,$s2 followed by sub $t3,$t0,$s4 have RAW? Can standard ALU forwarding solve it?

26 Control Hazards and Branch Prediction

Flushing wrong-path instructions

Slides Ch 7: 41-43 | Harris & Harris, pp. 426-428, 444-447

Branch dependence on PC

The next instruction address may depend on a branch outcome. A pipelined CPU often fetches sequential instructions before the branch decision is known. If the branch is taken contrary to prediction, wrong-path instructions must not alter architectural state.

Flushing

The course slide states its branch decision for beq is not known until pipeline stage four. Its diagram shows three sequential instructions fetched after a taken branch and then flushed. Determining the branch earlier reduces the potential penalty.

Prediction

A processor may predict taken/not-taken and later verify. Backward branches are often loop back-edges and are frequently taken; prediction using prior history can improve outcomes. A correct prediction avoids a misprediction flush.

Performance

A simplistic extra-CPI model multiplies branch fraction by misprediction fraction and penalty cycles, provided the assumptions explicitly match the problem. Base CPI and other stalls must be included if present.

Extra CPI from branch misses ≈ branch_fraction * miss_fraction * penalty_cycles

EXAM-READY DISTINCTION

Only wrong-path instructions are flushed; they do not commit state. Do not count every branch as a misprediction.

26 Control Hazards and Branch Prediction

Worked example and independent test

Slides Ch 7: 41-43 | Harris & Harris, pp. 426-428, 444-447

PROBLEM AND ASSUMPTIONS

Lecture-based: a taken beq is resolved in stage 4; and, or, sub after it were fetched, but target slt is the correct path.

Solution steps

1. When the branch resolves taken, the younger and/or/sub instructions represent the fall-through path and must be **flushed**.
2. The target-path slt begins fetching once the correct PC is selected; the example identifies three wrong-path instructions in this timing arrangement.
3. Original CPI extension: suppose 20% instructions are branches, 25% of those mispredict, with 3 wasted cycles each. Extra CPI = 0.20*0.25*3 = **0.15**.

Quantity	Assumption	Value
Branch fraction	20%	0.20
Mispredicted branch fraction	25% of branches	0.25
Penalty	3 cycles	3
Added CPI	0.20*0.25*3	0.15

WHAT THIS EXAMPLE PROVES

A 3-cycle penalty is tied to the depicted late-resolution example, not a universal penalty of every MIPS pipeline.

Try independently

If branch fraction is 10%, prediction accuracy 90%, and penalty 2 cycles, what extra CPI results?

Reference

Chapters 1, 2, 3, and 5: essentials you must retain

Lecture slides Ch 1/2/3/5; Harris & Harris printed pp. 9-280

Number representation

Concept	Rule / recognition cue
N-bit unsigned	0 ... 2^N-1; carry-out beyond N bits means unsigned overflow
N-bit two's complement	-2^(N-1) ... 2^(N-1)-1; negate by invert + 1
Signed addition overflow	Two operands share a sign; result sign differs
Hex conversion	Each hex digit is exactly four binary bits
Normalized IEEE single	1 sign; 8 exponent (bias 127); 23 fraction; implicit leading 1

Combinational logic

Concept	Reminder
De Morgan	(AB)'=A'+B'; (A+B)'=A'B'
SOP / POS	Canonical SOP uses output-1 minterms; POS uses output-0 maxterms
K-map	Gray-code order 00,01,11,10; power-of-two groups; edges wrap
MUX	2:1 Y=S'D0+SD1; 4:1 uses 2 selectors
Decoder	n inputs select one of 2^n outputs, subject to enable/polarity
Timing	t_pd along longest propagation path; t_cd along shortest contamination path

Sequential and building-block formulas

Concept	Rule
D flip-flop	Samples D only at active clock edge
Mealy / Moore	Output depends on state+input / state only
Period lower bound	t_clk-Q + t_combinational(max) + t_setup (ignore skew)
Full adder	S=A xor B xor Cin; Cout=AB+ACin+BCin
Memory capacity	number of words * bits per word; address bits = log2(words)

REFERENCE USE

Memorize what the formulas mean and when they apply. The sample exam stresses recognizing gate/logic behavior, signed or floating-point representations, FSMs, and circuit calculations.

Reference

Register conventions and instruction-field cheat sheet

Lecture slides Ch 6: 7-34, 44-87; Harris & Harris pp. 298-336

Register conventions

Register	Use	Saving rule
$zero	constant 0	not writable
$v0-$v1	return value	caller may need to save
$a0-$a3	function arguments	caller-save
$t0-$t9	temporaries	caller-save
$s0-$s7	saved variables	callee preserves
$sp	$sp points to stack	restore stack balance
$ra	return address	save before nested jal when needed

32-bit machine instruction fields

Format	Fields by width
R-type	op 6 | rs 5 | rt 5 | rd 5 | shamt 5 | funct 6
I-type	op 6 | rs 5 | rt 5 | immediate 16
J-type	op 6 | target 26

High-yield MIPS instructions

Instruction	Effect / address rule
add rd,rs,rt	rd = rs+rt
addi rt,rs,imm	rt = rs+signext(imm)
lw rt,off(rs)	rt = word at address rs+off
sw rt,off(rs)	word at address rs+off = rt
sll rd,rt,k	logical left shift by k
slt rd,rs,rt	rd=1 if rs<rt, else 0
beq / bne	branch if equal / not equal
j / jal / jr	jump / call saving return address / register jump

RECURRING TRAPS

Destination is first in add, but R-type field order places rd after rs and rt. lw writes a register; sw writes memory. Branch immediates are word offsets from PC+4; byte-array offsets and word-array offsets differ.

Reference

Control signals and processor performance

Lecture slides Ch 7: 9-43; Harris & Harris pp. 374-428

Simplified single-cycle control values

Control	lw	sw	R-type	Meaning
RegWrite	1	0	1	register write enable
MemWrite	0	1	0	data-memory write
ALUSrc	1	1	0	select immediate vs reg
RegDst	0	X	1	rt vs rd destination
MemtoReg	1	X	0	memory vs ALU result

Microarchitecture comparison

Design	Cycles/instruction	Clock limitation
Single-cycle	Exactly 1	Longest full instruction, usually lw
Multicycle	Varies by opcode	Slowest micro-operation
Pipelined	About 1 steady-state, absent stalls	Slowest pipeline stage + register overhead

Essential performance equations

CPU time = instruction count * average CPI * clock period

Average CPI = Σ (class fraction * cycles in class)

Ideal 5-stage pipeline cycles for N instructions = N + 4

Pipeline stages in order

Stage	Action
IF	Read instruction at PC
ID	Decode / read register file
EX	ALU operation / address calculation
MEM	Data read or write when needed
WB	Write result to register file when needed

TIMING WARNING

Single-cycle clock period is set by the slowest instruction. In a multi-cycle or pipelined design, compare CPI * cycle time and include hazards if the problem specifies them.

Reference

High-frequency mistakes and formulas at a glance

Synthesized from lecture slides and the uploaded sample exam

Choose the correct interpretation before calculating

Question cue	Use this rule
Two's-complement signed overflow	Same input signs + different result sign
Noise margin	NML=VIL-VOL; NMH=VOH-VIH
Four-variable K-map	Gray-code arrangement, wrap groups, 1/2/4/8 cells
Mealy vs Moore	Mealy output uses input+state; Moore only state
Memory address for A[i]	Base + i * element bytes
Endianness	Lowest address stores MSB (big) or LSB (little)
Load/store	lw reads memory into register; sw writes register into memory
R vs I vs J	6/5/5/5/5/6 ; 6/5/5/16 ; 6/26 bits
MIPS loop	Test condition, branch, body, update, jump
Recursive function	Save $ra and any needed callee-preserved registers
Single-cycle timing	Longest instruction path determines all cycles
Multicycle CPI	Frequency-weighted mean of instruction-class cycles
RAW forwarding	ALU result may bypass register-file write-back
Immediate load-use	Typically stall 1 cycle even with ordinary forwarding
Taken late branch	Flush younger wrong-path instructions

An open-notes workflow that saves time

- Mark where the instruction-format chart and control-signal table appear in this guide.
- Put one worked example of hex decoding, load/store, CPI, and pipeline hazards in separate tabbed positions.
- For multiple choice, first rule out answers that violate the exact register, bit-width, or timing convention stated.

DO NOT INFER FROM THE SAMPLE EXAM

The provided 4341 sample exam demonstrates multiple choice, true/false, and matching formats and includes sample answers. It does not establish the exact number or distribution of Exam 2 questions.

Practice

Exam-format questions across all six chapters

Original questions modeled on concepts and formats in the sample 4341 exam

Attempt without looking at the reference or answer page; write answers as question number + choice or T/F.

1. An 8-bit two's-complement value 11111111 is: A 255 | B -1 | C -128 | D +1
2. Which is signed overflow? A 100+45 in 8 bits | B -2+1 in 8 bits | C 2+3 in 8 bits | D 0+0
3. If F(A,B,C)=Σm(1,3,5,7), simplest F is: A A | B B | C C | D A+B
4. A 4:1 MUX with select 11 outputs: A D0 | B D1 | C D2 | D D3
5. A rising-edge D flip-flop updates Q: A whenever D changes | B at each rising edge | C whenever clock high | D only at reset
6. Moore FSM output depends on: A current state only | B input only | C state and input | D next instruction
7. A 256 x 16 memory has how many address bits? A 4 | B 8 | C 16 | D 256
8. The IEEE 754 single exponent field length is: A 5 | B 8 | C 23 | D 32
9. If $s0=0x10010020, lw $t0,8($s0) reads address: A 0x10010022 | B 0x10010028 | C 0x10010040 | D 8
10. R-type add has which funct number? A 0 | B 8 | C 32 | D 35
11. The single-cycle clock is set by: A most frequent instruction | B shortest path | C longest required path | D average CPI
12. A five-stage ideal pipeline runs 6 independent instructions in: A 5 cycles | B 6 | C 10 | D 30
13. Load-use immediately after lw commonly requires: A no hazard | B one stall | C 5 stalls | D branch flush
14. A taken branch resolved after younger fetches may require: A no action | B flushing wrong-path instructions | C changing ISA | D rewiring registers
15. A multicycle workload has 25% lw at 5 cycles and 75% R-type at 4 cycles. CPI = A 1 | B 4 | C 4.25 | D 5

True or false

16. A store word instruction writes its result into a general-purpose register.
17. A ripple-carry adder can have carry delay growing with bit width.
18. The first pipeline instruction ideally passes through IF, ID, EX, MEM, WB.
19. The ordinary MIPS branch immediate offset is counted in bytes from PC.

Practice

Answer key with brief reasoning

Original practice; verify weak topics on their numbered summary and example pages

#	Answer	Reason
1	B	All ones is -1 in two's complement.
2	A	Both inputs positive, but 145 lies outside signed 8-bit range.
3	C	Every 1-row has C=1; A and B vary.
4	D	11 selects data input 3.
5	B	D flip-flop samples at the selected edge.
6	A	Moore outputs depend only on current state.
7	B	256 = 2^8 words; eight address bits.
8	B	Single precision uses 1/8/23 bit fields.
9	B	Add 8 byte offset to the base address.
10	C	The R-type add funct field is decimal 32.
11	C	All instructions must fit in the worst instruction path.
12	C	N+4 = 6+4 = 10 ideal cycles.
13	B	Normal forwarding still cannot eliminate immediate load-use delay.
14	B	Wrong-path younger instructions must be flushed.
15	C	0.25(5)+0.75(4)=1.25+3=4.25.
16	False	sw changes memory and does not write a general-purpose register.
17	True	Each carry can depend on the preceding stage.
18	True	The conventional five-stage pipeline follows this sequence.
19	False	Branch immediate measures word offsets from PC+4.

HOW TO USE THESE RESULTS

A correct guess does not count as mastery. If any explanation seems unfamiliar, return to that topic's worked example and create a new version with different numerical values. Recheck your weak topics before exam day.

Sources

Scope, attribution, and limits

All source references are to materials supplied within this project

Primary course sources

- **Fall 2026 CS4341.004 syllabus**, p. 2: Exam 2 November 12; Chapters 1,2,3,5,6,7. Syllabus p. 3: Exam 2 weight 25%; exam policy says closed book and open notes.
- **Professor's lecture-slide PDFs**: Ch 1 From Zero to One (62 slides), Ch 2 Combinational Logic (67), Ch 3 Sequential Logic (65), Ch 5 Digital Building Blocks (70), Ch 6 Architecture (110), Ch 7 Microarchitecture (43). Slide numbers in topic headings are PDF slide positions.
- **eLearning Chapter 6 and Chapter 7 objective screenshots** (uploaded Oct. 22): prioritize C-to-MIPS, instruction decoding, arrays and functions; datapath tracing, performance comparison, RAW/control hazards and remedies.
- **Sample 4341 exam** (6 pages): formatting evidence for multiple-choice, true/false, and matching questions; not assumed to predict Exam 2 content.

Textbook

David Money Harris and Sarah L. Harris, Digital Design and Computer Architecture, 2nd ed. (Morgan Kaufmann, 2013), full PDF supplied in this project. Main chapters consulted: Ch 1 pp. 3-52; Ch 2 pp. 55-106; Ch 3 pp. 109-171; Ch 5 pp. 239-293; Ch 6 pp. 295-369; Ch 7 pp. 371-473. Page numbers refer to the **printed book**.

Example attribution and technical conventions

The worked examples are original teaching problems unless marked **lecture-derived** or **lecture-based**. Some use values also present in the course slides or sample exam (for example, R-type add 0x02328020, endianness of 0x23456789, IEEE representation 0xC0B00000, and multicycle CPI=4.12). Diagrams are newly drawn simplified conceptual graphics, not reproductions of slide schematics.

For MIPS, this guide follows the classic 32-bit MIPS model used in the supplied Chapter 6/7 slides: 32-bit instructions, a 5-stage IF/ID/EX/MEM/WB pipeline, and the noted instruction/address conventions. Performance examples use their specifically stated delays. Do not generalize cycle penalties, stage delays, or instruction subsets to unrelated architectures.

PRACTICAL SCOPE BOUNDARY

The course learning objectives and slides guided topic selection. Chapter 8 is excluded from this Exam 2 guide because the syllabus places it in Exam 3. No claim is made that every textbook subsection is examinable or that the sample exam gives an exact blueprint for Exam 2.